

Algoritmi e Strutture Dati – Parte A – 12/01/26

A1 – Complessità – Punti ≥ 8

Scrivere l'equazione di ricorrenza associata a questo algoritmo e calcolare la complessità computazionale che ne deriva.

```

int mystery(int[], int start, int end)
{
    if end == start then
        return A[start]
    else
        int i = end - start
        int sumSoFar = 0
        while i > 0 do
            sumSoFar += A[start + i]
            i = [i/2]
        int mid = [(start + end)/2]
        sumSoFar += mystery(A, start, mid)
        sumSoFar += mystery(A, mid + 1, end)
        return sumSoFar
}

```

Si assuma che la chiamata iniziale sia effettuata su un intervallo valido del vettore A .

A2 – Alberobello – Punti ≥ 10

Un *alberobello* è un albero binario in cui ogni nodo ha esattamente due figli oppure nessun figlio.

Dato un albero binario T , diciamo che un nodo t di T è una *radice alberobella* se il sottoalbero di T radicato in t è un alberobello, cioè se *tutti* i nodi di tale sottoalbero hanno zero o due figli.

Scrivere un algoritmo

```

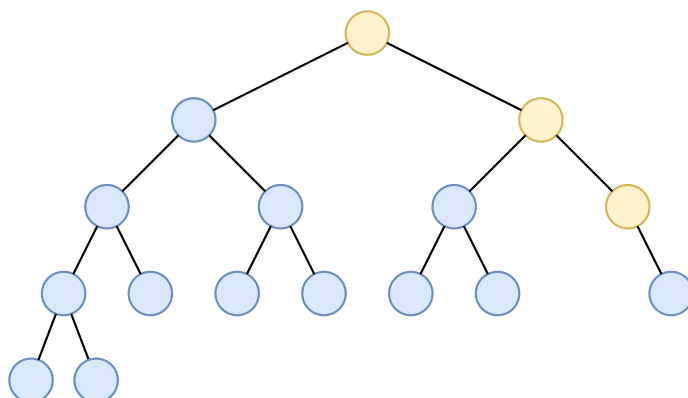
int countAB(TREE T)

```

che restituisca il numero di nodi di T che sono radici alberobelle.

Discutere brevemente la correttezza dell'algoritmo e la sua complessità computazionale.

Per esempio, nell'albero seguente, tutti i nodi evidenziati in azzurro sono radici alberobelle; pertanto, l'algoritmo deve restituire il valore 13.



A3 – Massimo raggiungibile – Punti ≥ 12

Scrivere un algoritmo

```

int[] maxReachable(GRAPH G)

```

che prende in input un grafo non orientato G , dove ogni nodo è dotato di un campo *value* che contiene un valore intero. L'algoritmo deve restituire un vettore M tale che, per ogni nodo u , $M[u]$ sia il massimo valore associato a un nodo v raggiungibile da u in G . Si ricorda che un nodo v è *raggiungibile* da un nodo u se esiste un cammino in G che parte da u e termina in v .

Discutere brevemente la correttezza dell'algoritmo e la sua complessità computazionale.

Algoritmi e Strutture Dati – Parte B – 12/01/26**B1 – Numeri zerabili – Punti ≥ 8**

Una stringa binaria S di lunghezza n (con eventuali zeri iniziali) è detta *zerabile* se contiene strettamente più zeri che uni. Scrivere un algoritmo

`printZerabile(int n)`

che stampa tutte le stringhe binarie zerabili di lunghezza esattamente n , in qualunque ordine.

Discutere brevemente la correttezza dell'algoritmo e la sua complessità computazionale. Algoritmi corretti che non sfruttano tecniche di pruning verranno valutati al massimo al 90%.

Per esempio, per $n = 4$ le stringhe zerabili sono: 0000, 0001, 0010, 0100, 1000.

B2 – Junior vs senior – Punti ≥ 10

Un'organizzazione no-profit è dotata di un insieme V contenente n volontari, divisi in due tipi:

- n_J volontari junior nell'insieme V_J ;
- n_S volontari senior nell'insieme V_S .

Si noti che $V = V_J \cup V_S$ e $V_J \cap V_S = \emptyset$.

L'organizzazione offre un insieme $S = \{s_1, \dots, s_m\}$ di servizi. Ogni volontario può essere assegnato ad al più un servizio e ha indicato un insieme $avail_i \subseteq S$ di servizi accettabili.

Per ogni servizio s_j sono dati:

- un numero esatto vt_j di volontari totali;
- un numero minimo vs_j di volontari senior.

Assumiamo $0 \leq vs_j \leq vt_j$.

Descrivere un algoritmo che verifichi se esiste un'assegnazione tale che ogni servizio s_j riceva esattamente vt_j volontari, di cui almeno vs_j senior, rispettando le richieste dei volontari.

B3 – Borsa – Punti ≥ 12

Sia $prices$ un vettore di n interi dove $prices[i]$ è il prezzo di una merce nel giorno i .

Una *operazione* è una coppia di giorni (b, s) tale che $0 \leq b < s \leq n - 1$:

- si compra la merce nel giorno b (buy);
- si rivende nel giorno s (sell);

- il guadagno dell'operazione è $prices[s] - prices[b]$.

Dovete eseguire esattamente k operazioni, con il vincolo che non si possono sovrapporre nel tempo: se $(b_1, s_1), (b_2, s_2), \dots, (b_k, s_k)$ sono le operazioni in ordine, allora

$$b_1 < s_1 < b_2 < s_2 < \dots < b_k < s_k.$$

Il guadagno totale è la somma dei guadagni delle singole operazioni.

Scrivere un algoritmo

`int maxGain(int[] prices, int n, int k)`

che restituisca il guadagno totale massimo ottenibile eseguendo esattamente k operazioni.

Discutere brevemente la correttezza dell'algoritmo e la sua complessità computazionale.

Esempio: per $prices = [1, 3, 6, 7, 4, 10]$ e $k = 2$, un guadagno massimo è

$$(7 - 1) + (10 - 4) = 12,$$

ottenuto comprando al giorno 0 a 1 e vendendo al giorno 3 a 7, poi comprando al giorno 4 a 4 e vendendo al giorno 5 a 10.